

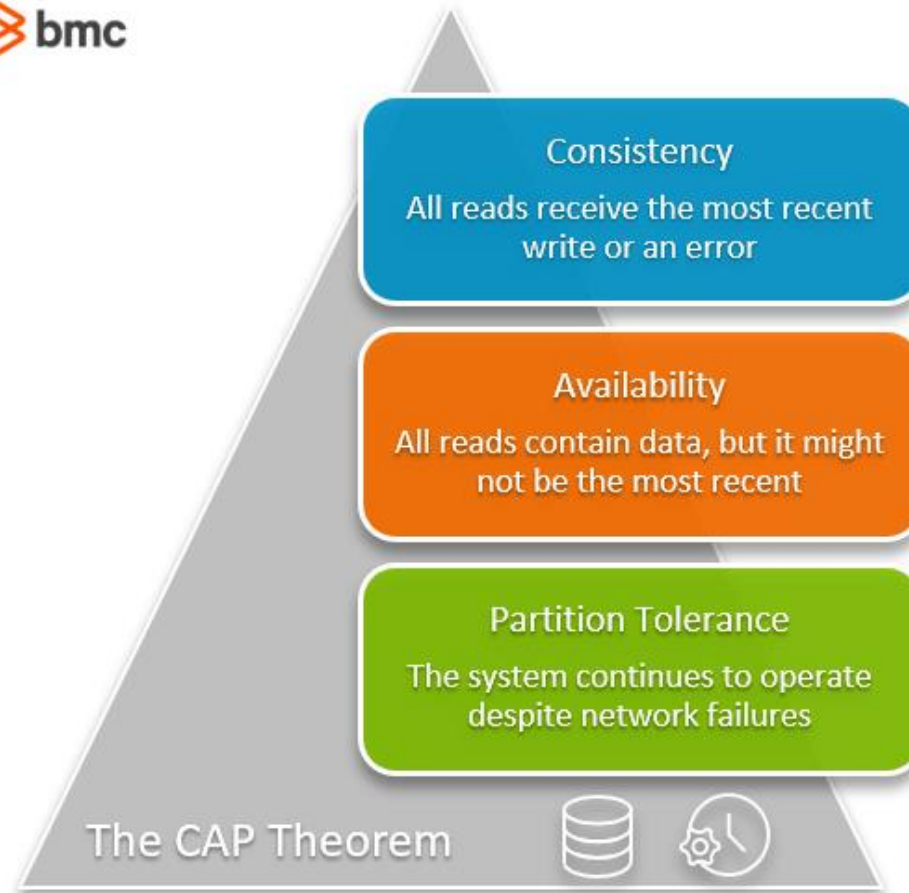
Advanced Data Management – Week 3

From the Relational Model to NoSQL: Denormalization and
Change of Thinking

CAP Theorem

- The CAP theorem is a fundamental theory of distributed systems and explains an unavoidable limitation in large-scale database design.
- It states that in a distributed data system it is impossible to guarantee simultaneously all three of the following properties:
 - Consistency
 - Availability
 - Partition Tolerance

CAP Theorem



What the CAP Theorem Says

- The CAP theorem states that:
- When a network partition occurs, a distributed system can guarantee either Consistency or Availability, but not both simultaneously.
- In other words:
- $C + P \rightarrow$ Availability is sacrificed
- $A + P \rightarrow$ Strong Consistency is sacrificed
- The third property cannot be maintained.

CAP – Relation with RDBMS and NoSQL

- Traditional RDBMS approach the CA model (Consistency + Availability) assuming reliable networks.
- NoSQL systems are usually designed as:
 - CP systems
 - AP systems
 - depending on the use case.
- In cloud and Big Data environments, Partition Tolerance is assumed, therefore the real choice becomes between Consistency and Availability.

C, P OR A Trade-off

- Most modern systems assume P is mandatory, therefore architecture focuses on C vs A trade-offs.
- Real Case Example

SYSTEM	CAP TYPE
MONGO DB	CP
CASSNDRA	AP
HBASE	CP
DYNAMO DB	AP

CAP Theorem – Conclusion

- The CAP theorem explains why:
 - NoSQL systems relax strict consistency
 - eventual consistency is acceptable
 - designing distributed databases involves conscious trade-offs

Understanding CAP is a necessary foundation for modern data architectures.

Course Roadmap & Architectural Thinking

NoSQL Categories (Overview)

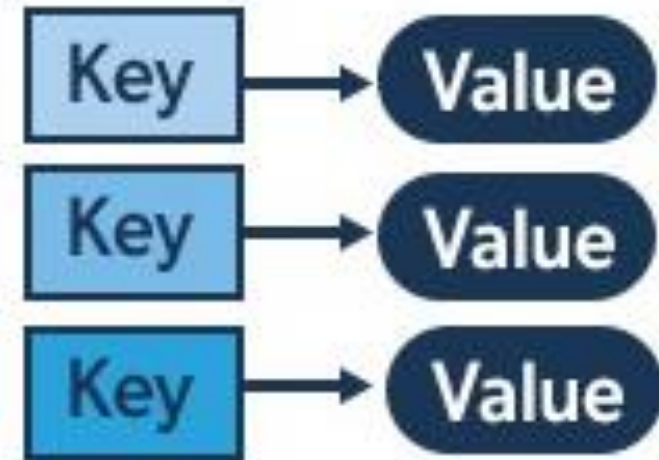
NoSQL systems are not a single technology but a family of different data models, each designed for specific workloads, access patterns, and relationships.

This classification helps explain why there is no “one database for everything.”

Key–Value Stores

- Data stored as key–value pairs
- Extremely simple model
- Very high performance and scalability
- Suitable for caches, sessions, counters

Key-Value



Document Databases

- Data stored as documents (JSON-like)
- Flexible schema
- Natural mapping to application objects
- Ideal for web and mobile applications

Document



Column-Family Stores

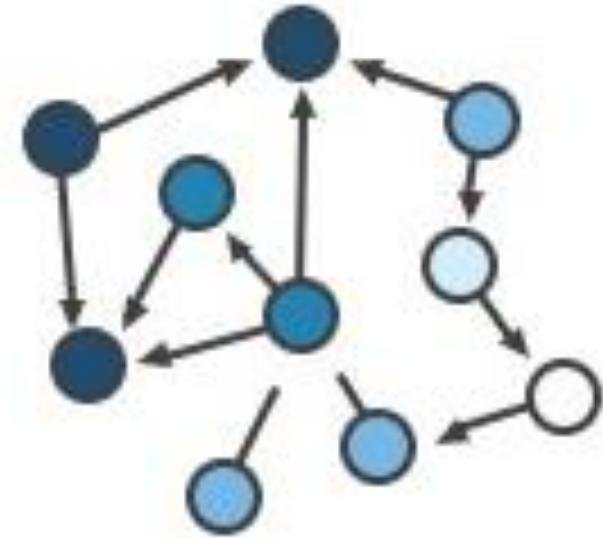
- Data organized into columns and column families
- Optimized for large-scale analytics
- High write throughput
- Common in Big Data platforms

Column-Family

Graph Databases

- Data represented as nodes and edges
- Focus on relationships
- Efficient graph traversal
- Suitable for social networks, recommendation systems, and knowledge graphs.

Graph



When Do We Use NoSQL

Using NoSQL is not the default choice and does not replace every database.

Instead, it results from a conscious architectural decision based on system requirements.

Not Always

- NoSQL systems are not a universal solution:
- not necessary for small applications
- not ideal for strict transactional systems
- not superior when strong consistency is required everywhere.

In many cases, a traditional RDBMS remains the correct choice.

Not Everywhere

- Even in large systems:
- NoSQL is not used in every subsystem
- different data require different storage technologies.

Modern practice is polyglot persistence.

With Architectural Thinking

- NoSQL is chosen when systems require:
- massive scale-out
- high concurrency
- low latency
- flexible schemas
- global distribution and high availability.

Database Choice as an Architectural Decision

- Database selection is not a technological trend nor an implementation detail.
- It is a core architectural decision affecting:
 - performance
 - scalability
 - reliability
 - system evolution.

Data Model

- The first fundamental question:
- how data are structured
- what entities and relationships exist
- whether the data model uses tables, documents, graphs, or time-series.

The data model must fit the problem naturally.

Access Patterns

- The database must be chosen based on:
- how data are read
- how they are updated
- which queries are critical.

We design for the queries that will actually occur.

Scalability Needs

- We must clearly answer:
- how much the system will grow
- whether scaling will be vertical or horizontal
- whether users will be globally distributed.

Scale is a design parameter, not an abstract concept.

Consistency Requirements

- Finally, it must be determined:
- whether strong consistency is required
- whether eventual consistency is acceptable
- which operations are critical and which are not.

Consistency is not a binary decision, but a spectrum of requirements depending on the use case.

Relational Data Model

- In the relational model:
- data are stored in tables, similar to an Excel sheet
- Columns represent the type of information stored
- Examples: Name, Email, Age
- Rows represent a complete record
- Example: one specific student
- Each row describes a real-world object.

Primary Key

- A Primary Key is a column (or combination of columns) that:
- uniquely identifies each row
- cannot be duplicated
- cannot be NULL
- Example:
- In a Students table, the column `student_id`.
- Purpose:
- To uniquely identify exactly one record.

Foreign Key

- A Foreign Key is a column that:
 - references the primary key of another table
 - creates a relationship between tables.
- Example:
 - Table Enrollments contains student_id referencing the Students table.
- Purpose:
 - To connect data without duplication.

Relationships Between Tables

- Relationships show **how data are connected**.
- Examples:
 - One student → many course enrollments
 - One order → one customer
- This design enables:
 - less duplicated data
 - better consistency
 - a cleaner database structure
- This principle is called **normalization**.

JOINS – Combining Data

- JOINS allow us to:
- combine data from **different tables**
- dynamically construct results during queries.
- Example:
- Student name from **Students**
Grade from **Grades**
- The data are **not stored together**, but combined dynamically.

Query-Centric Design

- The relational model is query-centric because:
 - Data are designed to:
 - support flexible queries
 - allow dynamic joins
 - answer many possible questions.
-
- In simple terms:
 - First we think what queries we want to run, then we design the tables.

Why the Relational Model Struggles at Scale

JOIN operations in distributed systems

- In a small system, all the tables are on the same server. A JOIN simply means: “read two tables from the same disk.”
- In a distributed environment, however:
 - one table might be on node A,
 - the other table on node B.
- To do a JOIN:
 - the data must be moved over the network,
 - synchronized,
 - and then joined.
- JOINS become expensive and slow.

Data Locality Problems

- Data locality means:
- The data required for a request should be stored close together.
- In relational systems:
- data are split across multiple tables
- queries require many joins.

Complex Node Coordination

- Relational systems rely on:
 - strong consistency
 - locks
 - transactions (ACID)
- In distributed systems this requires nodes to:
 - agree
 - lock data
 - wait for each other.
- As nodes increase, coordination becomes more difficult.

Increased Latency

- These factors lead to:
 - more network communication
 - more waiting
 - more synchronization.
-
- As a result the system responds more slowly, even for simple queries.

Conclusion

- The relational model struggles at scale because it relies on:
- joins
- strong consistency
- distributed coordination.

These work very well on a single server but become expensive in large distributed systems.

Σας Ευχαριστώ